

	UNIVERSIDAD DE SANTIAGO DE CHILE FACULTAD DE INGENIERÍA DEPARTAMENTO DE INGENIERÍA INFORMÁTICA	<i>U de Santiago de Chile</i>
--	---	-------------------------------

TALLER DE BASE DE DATOS DIURNO 1-2026

Laboratorio 3 — Bases de Datos NoSQL con MongoDB

Ayudante: Fernando Solís	Profesor: Matías Calderón
---------------------------------	----------------------------------

Enunciado Laboratorio 3

Entrega: 10 de agosto de 2026 (hasta las 12:00 hrs)

Implementación de Sistemas de Información con Arquitectura Multi-Capa, Bases de Datos NoSQL Avanzadas (MongoDB) y Seguridad JWT

1. Identificación del Curso

- **Asignatura:** Bases de Datos Avanzada
- **Nivel:** Pregrado / Ingeniería
- **Unidad Temática:** Integración de API REST, MongoDB Avanzado y Seguridad.

2. Requerimientos técnicos NoSQL

Para este laboratorio, se deben considerar los siguientes requisitos técnicos propios del motor documental:

- **Motor de Base de Datos:** MongoDB (versión 6.0 o superior), desplegado como Replica Set (mínimo un nodo primario y uno secundario), condición necesaria para poder utilizar Transacciones ACID multi-documento.
- **Validación de Esquemas:** Es obligatorio el uso de Schema Validation (\$jsonSchema) a nivel de colección, definiendo tipos de datos, campos requeridos, rangos válidos y reglas de negocio, dado que MongoDB no impone un esquema rígido por defecto.
- **Indexación:** Se debe diseñar y justificar una estrategia de índices adecuada para cada colección: índices compuestos para consultas frecuentes, índices únicos para restricciones de integridad, índices TTL para datos con expiración, e índices de texto cuando se requiera búsqueda por contenido.
- **Modelado de Datos:** Se debe justificar, para cada colección, la decisión de diseño entre “embeber” (embedding) y “referenciar” (referencing), de acuerdo con los patrones de acceso y consulta de cada dominio.
- **Capa de Servicios:** La API debe exponer y consumir documentos JSON, reflejando en sus respuestas el resultado de los Aggregation Pipelines diseñados para cada consulta compleja.

3. Objetivos de Aprendizaje

1. Diseñar esquemas de bases de datos NoSQL orientadas a documentos, aplicando correctamente los patrones de modelado embedding/referencing.
2. Implementar lógica de negocio persistente mediante Aggregation Pipelines, Transacciones multi-documento y Change Streams.
3. Desarrollar una capa de servicios (API) que garantice la integridad y seguridad de los datos mediante Schema Validation.
4. Gestionar la autenticación y autorización de usuarios mediante el estándar JSON Web Tokens (JWT).
5. Documentar técnicamente procesos de despliegue y consumo de servicios.

4. Requerimientos Técnicos (Arquitectura)

- **Motor de Base de Datos:** MongoDB.

- **Backend:** Desarrollo de una API REST utilizando lenguajes de programación de propósito general (ej. Python, JavaScript/TypeScript, Go, Java, C#).
- **Frontend:** Interfaz web funcional que consuma los recursos de la API (React, Vue, Angular o Vanilla JS).
- **Seguridad:** Implementación obligatoria de middleware de autenticación. Las rutas deben estar protegidas según el rol del usuario (RBAC).

A continuación, se presentan los 5 dominios de trabajo. Cada grupo debe modelar su base de datos en MongoDB (orientada a documentos) y construir los endpoints de la API para resolver las siguientes 6 tareas específicas, centradas en las capacidades avanzadas del motor: modelado, validación, transacciones, agregación, índices y reactividad.

Grupo 1: Sistema de Gestión de Torneos de Tiro con Arco

Contexto: Una plataforma para gestionar arqueros, categorías (ej. Recurvo, Compuesto), torneos y puntuaciones por ronda.

- **Modelado de Datos:** Justifique e implemente si las puntuaciones de cada ronda deben embeberse dentro del documento del torneo/arquero o referenciarse en una colección independiente, considerando el volumen de escritura durante una competencia en vivo.
- **Validación de Esquema:** Diseñe una regla de Schema Validation (\$jsonSchema) que impida registrar una puntuación fuera del rango permitido por el reglamento de la categoría, y que exija que el blanco y el arquero referenciados existan y pertenezcan al mismo torneo.
- **Transacción Multi-Documento:** Al registrar una puntuación, ¿cómo diseñaría una transacción que actualice de forma atómica el documento de la ronda, el acumulado del arquero y el ranking del torneo, revirtiendo todo el cambio si alguna operación falla?
- **Aggregation Pipeline:** Construya un pipeline (con \$group, \$sort y \$bucket) que calcule el promedio de puntaje y la mejor ronda por arquero, agrupado por categoría y torneo.
- **Índices:** Diseñe un índice compuesto único (arquero, torneo, ronda) que impida el doble registro de una puntuación en la misma ronda, y evalúe con explain() su impacto en el rendimiento.
- **Change Streams:** Implemente un listener sobre la colección de puntuaciones que, ante cada inserción, actualice en tiempo real una colección materializada de “ranking en vivo” del torneo (mediante \$merge).

Grupo 2: Sistema de Administración Académica Universitaria

Contexto: Una intranet para gestionar estudiantes, asignaturas, requisitos, semestres y calificaciones.

- **Modelado de Datos:** Justifique e implemente la decisión de embeber el historial de calificaciones dentro del documento del estudiante versus referenciarlo en una colección de inscripciones, considerando el crecimiento del documento a lo largo de la carrera.
- **Validación de Esquema:** Diseñe una regla de Schema Validation que impida crear una inscripción si el estudiante no cumple los requisitos (prerrequisitos aprobados) o si la sección ya alcanzó su cupo máximo.
- **Transacción Multi-Documento:** Para el proceso de inscripción, ¿cómo diseñaría una transacción que verifique cupo disponible, descuente el cupo de la sección y cree el documento de inscripción, garantizando que dos estudiantes no tomen el último cupo simultáneamente?
- **Aggregation Pipeline:** Construya un pipeline que calcule la tasa de aprobación y reprobación por asignatura y por semestre, utilizando \$group y \$bucket sobre las calificaciones finales.
- **Índices:** Diseñe un índice compuesto único (estudiante, sección, semestre) para impedir inscripciones duplicadas, y un índice de texto sobre el nombre de las asignaturas para el buscador del catálogo.
- **Vista Materializada / Change Streams:** Genere una colección materializada (\$merge) con el “certificado de notas” de cada estudiante, actualizada reactivamente mediante Change Streams cada vez que se registra una nueva calificación.

Grupo 3: Gestor de Clanes y Raids para MMORPG

Contexto: Una aplicación para gestionar jugadores, sus personajes, eventos (Raids), asistencia y distribución de botín (Loot).

- **Modelado de Datos:** Justifique e implemente si los personajes deben embeberse dentro del documento del jugador o referenciarse en una colección propia, considerando que cada personaje participa en múltiples raids de forma independiente.

- **Validación de Esquema:** Diseñe una regla de Schema Validation que impida asignar un ítem de Loot a un personaje que no participó en la Raid, o que ya se encuentre “caído” (fuera de combate) al momento de la distribución.
- **Transacción Multi-Documento:** En un evento de Raid, ¿cómo diseñaría una transacción que distribuya el Loot entre varios personajes de forma atómica, evitando que un mismo ítem quede asignado a más de un personaje ante solicitudes concurrentes?
- **Aggregation Pipeline:** Construya un pipeline que calcule el ranking de clanes según el desempeño en raids (tiempo de finalización, asistencia, daño total) usando \$group, \$lookup y \$sort.
- **Índices:** Diseñe índices compuestos para filtrar rápidamente personajes disponibles por clase y rol (ej. “Healer”) dentro de un clan, y un índice único sobre el nombre del personaje.
- **Change Streams:** Implemente un listener que, al detectarse la “muerte del Boss” en la colección de eventos de Raid, dispare automáticamente el proceso de distribución de Loot y actualice una colección materializada de “clanes mejor rankeados”.

Grupo 4: Plataforma de E-Commerce B2B

Contexto: Una tienda mayorista que maneja inventario de productos, clientes de empresas, carritos de compras y facturación.

- **Modelado de Datos:** Justifique e implemente si los ítems del carrito de compras deben embeberse dentro del documento del cliente/carrito o referenciarse contra el catálogo de productos, considerando la necesidad de mantener el precio histórico al momento de la compra.
- **Validación de Esquema:** Diseñe una regla de Schema Validation que impida agregar al carrito una cantidad mayor al stock disponible, o que no cumpla la cantidad mínima de pedido exigida para clientes B2B.
- **Transacción Multi-Documento:** Al procesar el Checkout, ¿cómo diseñaría una transacción que descuente el stock del producto, cree la orden y genere el documento de facturación de forma atómica, revirtiendo todo si el pago es rechazado?
- **Aggregation Pipeline:** Construya un pipeline que calcule el volumen de ventas proyectado por cliente y por categoría de producto, utilizando \$group, \$bucket y \$sort.
- **Índices:** Diseñe un índice único sobre el número de factura, un índice compuesto (cliente, fecha) para el historial de pedidos, y un índice TTL sobre los carritos abandonados para su limpieza automática.
- **Vista Materializada / Change Streams:** Genere una colección materializada (\$merge) con los “productos más vendidos”, actualizada reactivamente mediante Change Streams cada vez que se confirma una nueva orden.

Grupo 5: Panel de Control de Infraestructura Cloud

Contexto: Un sistema para que los usuarios desplieguen y gestionen instancias de servidores virtuales, bases de datos y consumo de ancho de banda.

- **Modelado de Datos:** Justifique e implemente si la configuración de cada instancia (CPU, RAM, disco) debe embeberse dentro del documento del servidor o referenciarse contra un catálogo de planes, considerando la frecuencia de cambios de configuración.
- **Validación de Esquema:** Diseñe una regla de Schema Validation que impida crear una nueva instancia si el cliente ya alcanzó el número máximo de servidores permitidos por su plan contratado.
- **Transacción Multi-Documento:** Al aprovisionar un nuevo servidor, ¿cómo diseñaría una transacción que cree el documento de la instancia y descuente el saldo disponible de la cuenta de facturación del cliente de forma atómica?
- **Aggregation Pipeline:** Construya un pipeline que calcule el consumo de ancho de banda y el costo asociado por cliente y por periodo de facturación, utilizando \$group y \$bucket sobre los registros de uso.
- **Índices:** Diseñe un índice TTL sobre la colección de métricas/logs de uso para su expiración automática, y un índice compuesto (cliente, estado) para consultar rápidamente los servidores activos.
- **Change Streams:** Implemente un listener que monitoree en tiempo real la colección de métricas y, al detectar que el consumo de un cliente supera el umbral de su plan, actualice una colección materializada de alertas y notifique al usuario.

5. Entregables y Normas de Envío

Para que el proyecto sea considerado como entregado y válido para evaluación, cada grupo deberá proporcionar un repositorio de GitHub (o GitLab/Bitbucket) que contenga la siguiente estructura y elementos:

6. Código Fuente: Organizado en carpetas claras para backend y frontend.
7. Script(s) de Base de Datos: Archivo(s) de definición (ej. scripts .js para mongosh, o scripts de seed) con la creación de colecciones, validadores JSON Schema, índices (compuestos, únicos, TTL, de texto según corresponda), definición de pipelines de agregación materializados (\$merge) e inserción de datos de prueba (seeders).
8. README.md (Documentación Principal): Este archivo es el corazón de la entrega. Debe incluir arquitectura y tecnologías utilizadas; manual de instalación y despliegue paso a paso (variables de entorno, comandos para levantar contenedores Docker o instalar dependencias, incluyendo la configuración del Replica Set de MongoDB); y documentación de la API (endpoints principales y ejemplos de JSON).
9. Carpeta Presentación: Debe incluir las diapositivas (PDF o PPTX) que se utilizarán para la defensa del proyecto.

Regla Crítica de Evaluación (Criterio de Aceptación)

Funcionabilidad del Manual de Instalación: Es responsabilidad estricta de los alumnos asegurar que el sistema pueda ser desplegado siguiendo exclusivamente las instrucciones detalladas en el archivo README.md.

Si al momento de seguir el manual de instalación el sistema no funciona, no compila o presenta errores de configuración insalvables, el proyecto será calificado automáticamente con la nota mínima. Se recomienda probar el proceso de instalación desde cero en una máquina limpia antes de la entrega final.

Método de evaluación: La nota final de cada laboratorio corresponderá al promedio simple entre la nota de presentación y la nota de código. Sin embargo, para que la nota de presentación sea considerada, la nota de código debe ser igual o superior a 4,0. En caso contrario, la nota final corresponderá únicamente a la nota de código, resultando en reprobación de ese laboratorio.